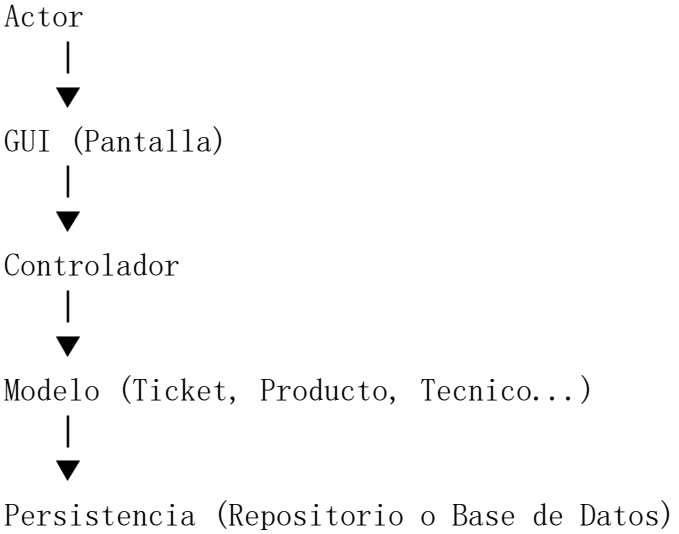
DIAGRAMA DE SECUENCIAS

Por lo que contás, quiere que apliquen algo parecido a **MVC (Modelo-Vista-Controlador)**, que es muy común en proyectos de C++ con interfaces gráficas.



Yo agregaría estos objetos

Para el primer diagrama:

Cliente (Actor)

GUIRegistrarTicket

TicketController

TecnicoController

Ticket

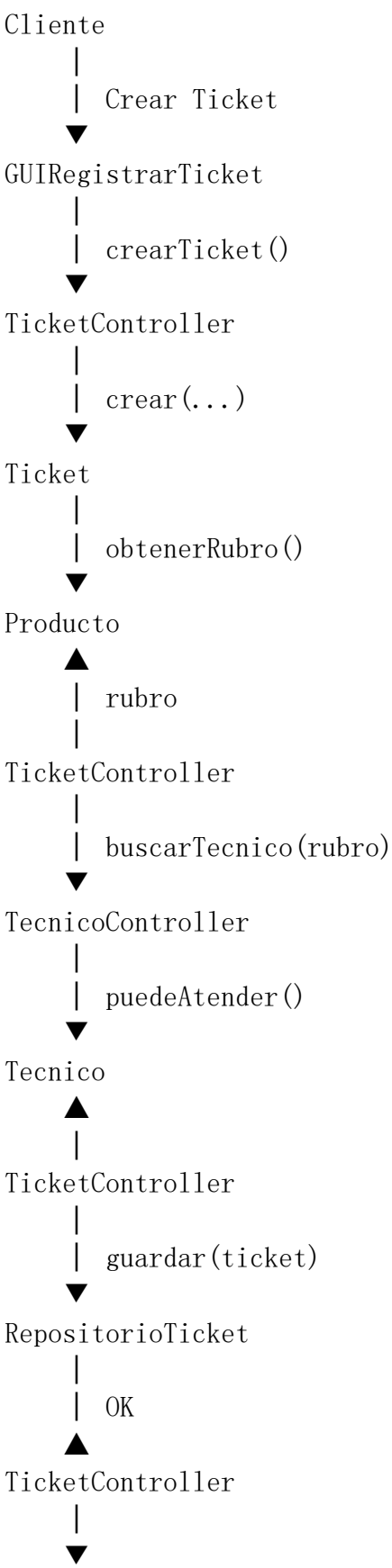
Producto

Tecnico

RepositorioTicket

Si después usan archivos (fstream) o una base de datos, el repositorio representa esa capa.

El primer diagrama quedaría así



GUIRegistrarTicket



Cliente

Fijate que aparecen claramente:

- GUI
- Controladores
- Modelo
- Persistencia

Eso es exactamente lo que suelen pedir.

Segundo diagrama

Agregar un Hito

Ahí usaría:

Tecnico

GUITicket

HitoController

Ticket

Hito

RepositorioTicket

NotificacionController

La secuencia sería:

Tecnico

↓

GUITicket

↓

HitoController

↓

Ticket

↓

crearHito()

↓

RepositorioTicket

↓

guardar()

↓

NotificacionController

↓

Cliente

Notificaciones

Como el profesor mencionó **controladores de notificaciones**, yo sí agregaría uno.

NotificacionController

C++
enviarNotificacion(cliente)

En el diagrama aparece después de guardar el ticket.

RepositorioTicket

↓

NotificacionController

↓

Cliente

Persistencia

Yo no pondría directamente

Archivo
O
BaseDatos

Porque el día de mañana puede cambiar.

Pondría algo como:

RepositorioTicket

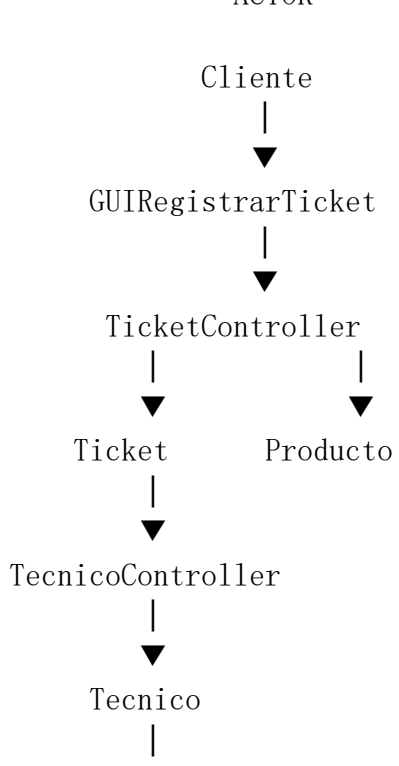
O TicketDAO

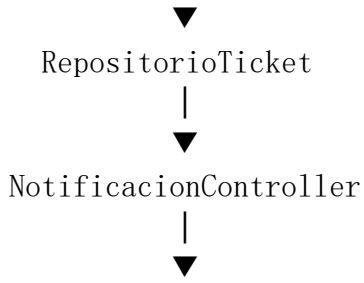
Y luego ese repositorio internamente escribe en:

- archivo
- SQLite
- MySQL

Lo importante para UML es que exista una capa de persistencia.

Arquitectura que yo seguiría





Cliente

Mi recomendación

Yo **no haría los diagramas de secuencia sólo con** `Ticket`**,** `Cliente` **y** `Técnico`, porque con el comentario del profesor quedarían incompletos.

En cambio, haría los diagramas con **cinco capas bien diferenciadas**:

1. **Actor** (Cliente o Técnico).
2. **GUI** (pantalla o formulario).
3. **Controladores** (`TicketController`, `TecnicoController`, `NotificacionController`, `HitoController`).
4. **Modelo** (`Ticket`, `Producto`, `Tecnico`, `Hito`).
5. **Persistencia** (`RepositorioTicket` o `TicketDAO`).

Ese enfoque demuestra una arquitectura limpia y orientada a objetos, y suele ser exactamente lo que esperan cuando piden considerar la GUI, los controladores y la persistencia en los diagramas de secuencia.

Ejemplo con tu proyecto

Supongamos el caso **Crear Ticket**.

Con PlantUML escribirías:

@startuml

actor Cliente

boundary GUIRegistrarTicket

control TicketController

control TecnicoController

entity Ticket

entity Producto

entity Tecnico

database RepositorioTicket

control NotificacionController

Cliente -> GUIRegistrarTicket : Completa formulario

GUIRegistrarTicket -> TicketController : crearTicket(datos)

TicketController -> Producto : obtenerRubro()

Producto --> TicketController : RubroTipo

TicketController -> TecnicoController : buscarTecnico(rubro)

TecnicoController -> Tecnico : puedeAtender(rubro)

Tecnico --> TecnicoController : disponible

TecnicoController --> TicketController : tecnico

TicketController -> Ticket : crear()

TicketController -> RepositorioTicket : guardar(ticket)

RepositorioTicket --> TicketController : OK

TicketController -> NotificacionController : notificar(cliente)

NotificacionController --> Cliente : Ticket registrado

@enduml

Además podés usar fragmentos UML

Por ejemplo, si no hay técnicos disponibles:

alt Técnico disponible

TicketController -> Tecnico : asignar()

else No hay técnicos

TicketController -> GUIRegistrarTicket : Mostrar error

End

Eso queda muy profesional.

También podés representar la persistencia

database RepositorioTicket

O database BaseDeDatos

Para la GUI

PlantUML tiene estereotipos UML:

boundary GUIRegistrarTicket

Aparece como una interfaz gráfica.

Para los controladores

control TicketController

control HitoController

control NotificacionController

Se distinguen visualmente de las entidades.

Para las entidades

entity Ticket

entity Producto

entity Tecnico

entity Hito

Quedaría una arquitectura muy clara

Actor

|

▼

Boundary (GUI)

|

▼

Controller

|

▼

Entities

|

▼

Database

Eso es justamente lo que suelen buscar cuando un profesor dice:

> "Tengan en cuenta la GUI, los controladores y la persistencia."

Mi propuesta

Ya que conozco tu trabajo (requerimientos, casos de uso y UML), te propongo hacer los diagramas **como si fueran de un proyecto profesional en C++ con patrón MVC**.

Haríamos dos diagramas:

1. Crear Ticket y asignar Técnico por Rubro

Con:

- Actor (Cliente)
- GUIRegistrarTicket
- TicketController
- TecnicoController
- Ticket
- Producto
- Tecnico
- RepositorioTicket
- NotificacionController

2. Registrar un Hito y notificar

Con:

- Técnico
- GUITicket
- HitoController
- Ticket
- Hito
- RepositorioTicket
- NotificacionController

Creo que esta es la mejor opción para tu entrega. No serían diagramas "académicos básicos", sino diagramas de secuencia con una arquitectura por capas (GUI → Controladores → Modelo → Persistencia), que es exactamente el enfoque que suele esperarse cuando el profesor pide considerar la interfaz gráfica, los controladores y la persistencia. Además, PlantUML permite representarlo de forma muy limpia y profesional.

Yo escribiría el primer diagrama así

Cliente

↓

GUIRegistrarTicket

↓

TicketController

↓

Producto

↓

TecnicoController

↓

Tecnico

↓

Ticket

↓

RepositorioTicket

↓

NotificacionController

↓

Cliente

Ese flujo representa exactamente el caso de uso.

Para el segundo diagrama

Haría:

Tecnico

↓

GUITicket

↓

HitoController

↓

Ticket

↓

Hito

↓

RepositorioTicket

↓

NotificacionController

↓

Cliente

Mi recomendación

Yo **separaría los dos diagramas** y los nombraría, por ejemplo:

- **DS-01 – Registrar Ticket y Asignar Técnico por Rubro**
- **DS-02 – Registrar Hito y Notificar al Cliente**

Además, haría un pequeño ajuste arquitectónico: el `TicketController` debería interactuar principalmente con el objeto `Ticket`, y el `Ticket` debería asociarse con el `Producto`. La búsqueda del técnico se realiza utilizando el rubro del producto ya creado. Ese flujo refleja mejor el modelo de clases que diseñaste y mantiene una coherencia entre el UML estático y los diagramas de secuencia.

Con esos cambios, los diagramas quedarían con un nivel muy bueno para una entrega de la materia.

PlantUML necesita instrucciones con la sintaxis UML. Por ejemplo:

```
@startuml

actor Cliente

boundary GUIRegistrarTicket

control TicketController

control TecnicoController

control NotificacionController

entity Ticket

entity Producto

entity Tecnico

database RepositorioTicket

Cliente -> GUIRegistrarTicket : Completa formulario

GUIRegistrarTicket -> TicketController : crearTicket(datos)

TicketController -> Producto : crear(datosProducto)

Producto --> TicketController : Producto

TicketController -> Ticket : crear(producto)
```

```
TicketController -> TecnicoController : buscarTecnico(producto.rubro)

alt Técnico disponible

    TecnicoController -> Tecnico : puedeAtender(rubro)

    Tecnico --> TecnicoController : disponible

    TecnicoController --> TicketController : técnico

    TicketController -> Ticket : asignarTecnico(tecnico)

    TicketController -> RepositorioTicket : guardar(ticket)

    RepositorioTicket --> TicketController : OK

    TicketController -> NotificacionController : notificar(cliente)

    NotificacionController --> Cliente : Ticket registrado

else No hay técnicos disponibles

    TecnicoController --> TicketController : sin técnico

    TicketController --> GUIRegistrarTicket : Mostrar mensaje

end

@enduml
```

¿Por qué este diagrama es mejor?

Porque sigue exactamente la arquitectura que te pidió el profesor:

```
Cliente
   |
GUI
   |
Controladores
   |
Modelo
   |
Persistencia
   |
Notificación
```

Además:

- ✔ Aparece la GUI (`GUIRegistrarTicket`).
- ✔ Aparecen los controladores.
- ✔ Aparecen las entidades (`Ticket`, `Producto`, `Tecnico`).
- ✔ Aparece la persistencia (`RepositorioTicket`).
- ✔ Aparece un flujo alternativo (`alt`) cuando no hay técnicos disponibles.

Mi recomendación

Como ya conozco **tu UML**, **los requerimientos** y **el comentario del profesor**, no improvisaría los diagramas.

Haría **dos diagramas de secuencia completos y consistentes con tu diseño**:

1. **DS-01 – Crear Ticket y Asignar Técnico por Rubro.**
2. **DS-02 – Registrar Hito, Guardar Cambios y Notificar al Cliente.**

Los prepararía con una arquitectura por capas (GUI → Controladores → Modelo → Persistencia), usando buenas prácticas de PlantUML (activaciones, mensajes de retorno y fragmentos `alt`/`opt`). Así te quedarían con un nivel profesional y totalmente alineados con el UML que ya hiciste. Creo que esa sería la mejor versión para entregar.

DS-01 - Crear Ticket y Asignar Técnico por Rubro

```
@startuml
title DS-01 - Crear Ticket y Asignar Técnico por Rubro

actor Cliente

boundary GUIRegistrarTicket
control TicketController
control TecnicoController
entity Producto
entity Ticket
entity Tecnico
database RepositorioTicket
control NotificacionController

Cliente -> GUIRegistrarTicket : Completa formulario\n(Descripción + Producto)

activate GUIRegistrarTicket

GUIRegistrarTicket -> TicketController : crearTicket(datos)
```

```
activate TicketController

TicketController -> Producto : crear(datosProducto)
activate Producto
Producto --> TicketController : Producto
deactivate Producto

TicketController -> Ticket : crear(descripcion, producto)
activate Ticket
Ticket --> TicketController : Ticket creado
deactivate Ticket

TicketController -> TecnicoController : buscarTecnico(producto.rubro)

activate TecnicoController

TecnicoController -> Tecnico : puedeAtender(producto.rubro)

activate Tecnico

alt Técnico disponible

    Tecnico --> TecnicoController : Disponible
    TecnicoController --> TicketController : Técnico encontrado

    TicketController -> Ticket : asignarTecnico(tecnico)

    TicketController -> RepositorioTicket : guardar(ticket)
    activate RepositorioTicket
    RepositorioTicket --> TicketController : OK
    deactivate RepositorioTicket

    TicketController -> NotificacionController :
notificarCliente(ticket)

    activate NotificacionController
    NotificacionController --> Cliente : Ticket registrado
    deactivate NotificacionController

else No hay técnicos disponibles

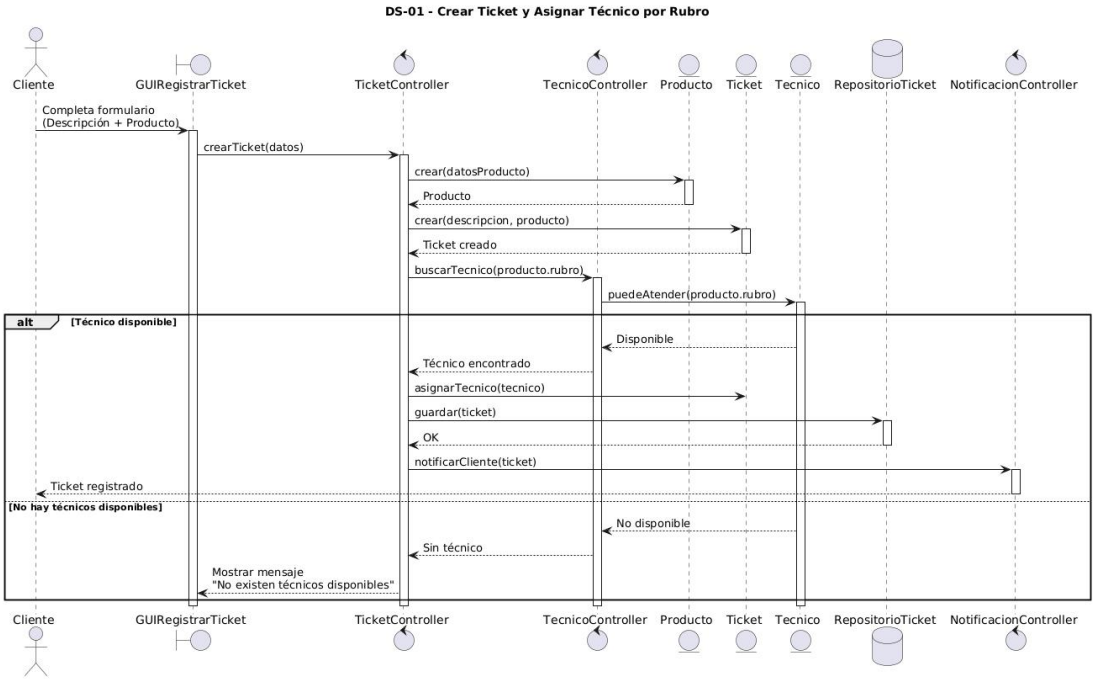
    Tecnico --> TecnicoController : No disponible
    TecnicoController --> TicketController : Sin técnico
```

```
    TicketController --> GUIRegistrarTicket : Mostrar mensaje\n"No existen técnicos disponibles"

end

deactivate Tecnico
deactivate TecnicoController
deactivate TicketController
deactivate GUIRegistrarTicket

@enduml
```



DS-02 - Registrar Hito, Guardar Cambios y Notificar al Cliente

```
@startuml
title DS-02 - Registrar Hito, Guardar Cambios y Notificar al Cliente

actor Tecnico

boundary GUITicket
control HitoController
entity Ticket
entity Hito
database RepositorioTicket
control NotificacionController
```

```
actor Cliente

Tecnico -> GUITicket : Registrar avance

activate GUITicket

GUITicket -> HitoController : agregarHito(ticket, mensaje)

activate HitoController

HitoController -> Ticket : obtenerTicket()

activate Ticket

Ticket -> Hito : crear(mensaje)

activate Hito

Hito --> Ticket : Hito creado

deactivate Hito

Ticket --> HitoController : Ticket actualizado

deactivate Ticket

HitoController -> RepositorioTicket : guardar(ticket)

activate RepositorioTicket

RepositorioTicket --> HitoController : OK

deactivate RepositorioTicket

HitoController -> NotificacionController : notificarCliente()

activate NotificacionController

NotificacionController --> Cliente : Nuevo avance del Ticket

deactivate NotificacionController

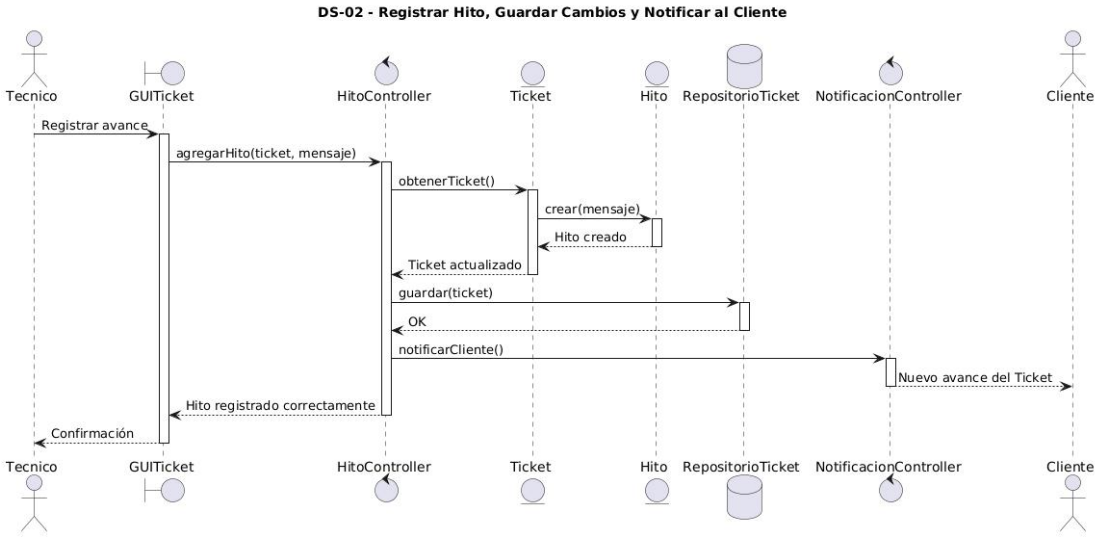
HitoController --> GUITicket : Hito registrado correctamente
```

```
deactivate HitoController

GUITicket --> Tecnico : Confirmación

deactivate GUITicket

@enduml
```



¿Por qué estos diagramas están bien?

Siguen exactamente la arquitectura que tu profesor pidió:

```
Actor
  |
  ▼
GUI (Boundary)
  |
  ▼
Controladores
  |
  ▼
Modelo (Entities)
  |
  ▼
Persistencia (Repository)
  |
  ▼
Notificaciones
```

Además, son coherentes con el UML que hiciste:

- `Ticket`
- `Producto`
- `Tecnico`
- `Hito`
- `Cliente`
- `Usuario` (implícito porque Cliente y Técnico son actores en cada caso)

Un pequeño detalle para mejorar la entrega

En tu UML el caso de uso habla de **mensajes** y en el diagrama de clases la entidad se llama **Hito**. Para mantener consistencia, usá siempre el mismo término en todos los documentos. Si en el UML la clase es `Hito`, entonces en el diagrama de secuencia conviene que el controlador use `agregarHito(...)` y la entidad sea `Hito`, como quedó en el código.

Mi sugerencia adicional

Si querés destacarte en la entrega, podés agregar un **tercer diagrama opcional**:

- **DS-03 – Cerrar/Reabrir Ticket**, mostrando cómo el cliente solicita el cierre, el controlador cambia el estado, se guarda en el repositorio y se notifica al técnico o al cliente.

No suele ser obligatorio, pero demuestra que modelaste el ciclo completo del ticket y complementa muy bien los dos diagramas principales.

Yo lo llamaría:

```
DS-03 - Cerrar/Reabrir Ticket
```

Este diagrama sigue exactamente la misma arquitectura que los otros dos (GUI → Controladores → Modelo → Persistencia → Notificaciones), por lo que tu trabajo queda consistente.

Código PlantUML

```
@startuml
title DS-03 - Cerrar o Reabrir Ticket

actor Cliente

boundary GUITicket
control TicketController
entity Ticket
```

```
database RepositorioTicket
control NotificacionController
actor Tecnico

Cliente -> GUITicket : Solicita cerrar/reabrir Ticket

activate GUITicket

GUITicket -> TicketController : cambiarEstado(idTicket, nuevoEstado)

activate TicketController

TicketController -> Ticket : cambiarEstado(nuevoEstado)

activate Ticket

Ticket --> TicketController : Estado actualizado

deactivate Ticket

TicketController -> RepositorioTicket : guardar(ticket)

activate RepositorioTicket

RepositorioTicket --> TicketController : OK

deactivate RepositorioTicket

alt Ticket cerrado

    TicketController -> NotificacionController :
notificarCierre(ticket)

    activate NotificacionController

    NotificacionController --> Cliente : Ticket cerrado correctamente
    NotificacionController --> Tecnico : Ticket finalizado

    deactivate NotificacionController

else Ticket reabierto

    TicketController -> NotificacionController :
notificarReapertura(ticket)
```

```
    activate NotificacionController

    NotificacionController --> Cliente : Ticket reabierto
    NotificacionController --> Tecnico : Ticket reasignado

    deactivate NotificacionController

end

TicketController --> GUITicket : Operación exitosa

deactivate TicketController

GUITicket --> Cliente : Confirmación

deactivate GUITicket

@enduml
```

Con estos tres diagramas cubrís prácticamente todo el sistema

DS-01

📌 Registrar Ticket y asignar Técnico por Rubro

- Cliente
- GUI
- TicketController
- Producto
- Técnico
- Persistencia
- Notificaciones

DS-02

📌 Registrar Hito

- Técnico
- GUI
- HitoController
- Ticket
- Hito
- Persistencia

- Notificaciones

DS-03

📌 Cerrar/Reabrir Ticket

- Cliente
- GUI
- TicketController
- Ticket
- Persistencia
- Notificaciones
- Técnico

Mi última recomendación (como si fuera tu profesor)

Viendo todo el trabajo que hicimos, yo haría un pequeño ajuste de nombres para que todo quede perfectamente consistente:

UML	Diagrama de Secuencia
Usuario	Actor Cliente / Actor Técnico
Cliente	Cliente
Técnico	Técnico
Producto	Producto
Ticket	Ticket
Hito	Hito
RubroTipo	producto.rubro
RepositorioTicket	RepositorioTicket
TicketController	TicketController
HitoController	HitoController
NotificacionController	NotificacionController

De esa manera, cualquier profesor puede comparar el diagrama de clases con los diagramas de secuencia y ver que todos utilizan exactamente las mismas clases y responsabilidades.

Mi impresión final

Con el UML actualizado, los requerimientos, los casos de uso y estos tres diagramas de secuencia, tu trabajo queda muy completo. No sólo resolvés la modificación pedida por el cliente (Producto y Rubro para especializar técnicos), sino que además mostrás una arquitectura limpia con GUI, controladores, entidades, persistencia y notificaciones. Ese nivel de detalle suele marcar una diferencia positiva en una entrega evaluativa. ¡Mucha suerte con la presentación!

DS-03 - Cerrar/Reabrir Ticket

